

Pádel & Gol

Sistema de Gestión de Reservas Deportivas
Informe de desarrollo—TP1 y TP2
Morán Santiago / Fakiani Octavio | AW2 — IES21 — 2026

TP1 — Propuesta del proyecto

¿De qué se trata el proyecto?

El proyecto es una página web para gestionar reservas de canchas deportivas. La idea surgió porque muchos complejos de pádel y fútbol siguen usando WhatsApp o papel para anotar los turnos, lo que genera errores y confusión. Con esta plataforma los usuarios pueden ver las canchas disponibles y consultar sus reservas de forma más ordenada.

¿Qué incluye el sistema?

- Ver canchas disponibles con sus horarios
- Consultar reservas registradas
- Panel de administración (a implementar en etapas siguientes)

Equipo

- Morán Santiago: Front-End y Testing
- Fakiani Octavio: Back-End e Integración

Herramientas que se van a usar

- Node.js y Express para el servidor
- HTML, CSS y JavaScript para el front-end
- PostgreSQL como base de datos (a partir del TP3)
- Git y GitHub para versionar el código
- Trello para organizar las tareas del equipo

TP2 — Servidor con Express y conexión a la API

¿Qué se hizo en este TP?

Se creó un servidor con Node.js y Express que es el que sirve el sitio web. Cuando alguien abre la página en el navegador, es el servidor el que le manda los archivos HTML, CSS y JavaScript. Además, el servidor se conecta a MockAPI para traer los datos de las canchas y las reservas.

¿Cómo funciona?

El usuario abre el navegador y entra a la página. El front-end hace un pedido al servidor (por ejemplo a /canchas), el servidor va a buscar esos datos a MockAPI y se los devuelve al navegador para mostrarlos en pantalla. El usuario nunca habla directamente con MockAPI, siempre pasa por el servidor.

Páginas que tiene el sitio

- Inicio: presentación del sistema con acceso rápido a las canchas
- Canchas: lista todas las canchas con deporte, precio y disponibilidad
- Reservas: muestra las reservas con fecha, hora y estado

Estructura del proyecto

El código quedó organizado en dos partes principales: el archivo `index.mjs` que es el servidor, y la carpeta `front/` que tiene todos los archivos del sitio web. Dentro de `front/` están los HTMLs, los estilos en CSS y los scripts de JavaScript separados por página.

Rutas del servidor

- GET `/canchas`: devuelve la lista de canchas desde MockAPI
- GET `/canchas/:id`: devuelve una cancha específica por su ID
- GET `/reservas`: devuelve la lista de reservas desde MockAPI

¿Cómo se corre el proyecto?

Dentro de la carpeta del proyecto se ejecuta `npm install` para instalar las dependencias y después `npm run dev` para levantar el servidor. Luego se abre el navegador en `http://localhost:3000` y ya se puede ver el sitio funcionando.

Dificultades que surgieron

Al separar los scripts en archivos individuales por página, dejaron de funcionar porque estaban en una carpeta fuera de `front/` y Express no los podía servir. Se solucionó moviendo la carpeta `scripts/` adentro de `front/` y corrigiendo las rutas en los archivos HTML.

Pádel & Gol

Sistema de Gestión de Reservas Deportivas

Informe de desarrollo — TP3

Morán Santiago / Fakiani Octavio | AW2 — IES 21 — 2026

¿Qué se hizo en este TP?

En el TP3 reemplazamos los datos que veníamos usando desde MockAPI por una base de datos real. Se armó el CRUD completo de canchas (alta, baja, modificación y dos lecturas) y se conectó todo a una base de datos PostgreSQL alojada en Supabase.

¿Por qué Supabase?

Supabase nos da una base de datos PostgreSQL gratuita en la nube, sin tener que instalar nada localmente. Solo hace falta crear el proyecto en Supabase y copiar la cadena de conexión para que el servidor se conecte a la base.

¿Cómo se conecta el servidor a la base?

El servidor (Node.js + Express) se conecta directamente a la base de datos de Supabase usando el paquete pg, con un Pool de conexiones. La cadena de conexión (DATABASE_URL) se guarda en un archivo .env que no se sube al repositorio, así no quedan expuestas las credenciales. Al iniciar el servidor, se verifica la conexión y se crea la tabla canchas si todavía no existe.

Estructura del proyecto (patrón MVC)

- config/db.mjs: configura la conexión a Supabase
- models/Cancha.mjs: tiene las consultas SQL a la tabla canchas
- controllers/canchaController.mjs: recibe la petición y decide qué responder
- routes/: define las rutas y las conecta con el controlador
- front/: HTML, CSS y JS del sitio y del panel de administración

Endpoints de la API

API para administrar (/api/canchas):

- GET /api/canchas: lista todas las canchas
- GET /api/canchas/:id: trae una cancha puntual
- POST /api/canchas: crea una cancha nueva
- PUT /api/canchas/:id: actualiza una cancha
- DELETE /api/canchas/:id: elimina una cancha

API pública de solo lectura (/api/web), para el sitio:

- GET /api/web/canchas: lista las canchas para mostrar en la web
- GET /api/web/canchas/:id: detalle de una cancha

Panel de administración

Se armó una página (admin.html) donde se pueden ver todas las canchas en una tabla, agregar una nueva con un formulario, editar una existente con un modal, y eliminarla con confirmación. Todo esto consume la API CRUD que mencionamos arriba.

Manejo de errores

Se usa try/catch tanto en el modelo como en el controlador. El modelo atrapa los errores de las consultas a la base de datos y los muestra en consola con el nombre de la función donde ocurrieron, y el controlador decide qué mensaje y código de error devolver al cliente (por ejemplo, 404 si la cancha no existe, o 400 si los datos no son válidos).

Corrección que hicimos después de la devolución del profe

Al principio, el front-end de la página de canchas intentaba conectarse directamente a Supabase desde el navegador, y solo usaba nuestra propia API si esa conexión fallaba. Lo corregimos para que el front-end use siempre la API que armamos nosotros, y que sea el servidor (no el navegador) el que hable con Supabase. Así seguimos la arquitectura que se pidió: front-end → API propia → base de datos.

¿Cómo se corre el proyecto?

Hay que crear un archivo .env con la URL de conexión a Supabase (DATABASE_URL) y el puerto. Después, dentro de la carpeta del proyecto:

```
npm install
```

```
npm run dev
```

Y se abre el navegador en <http://localhost:3000>

Qué queda pendiente

Las reservas todavía no se migraron a la base de datos, por ahora solo está armado el CRUD de canchas. La idea es sumar la tabla de reservas y su lógica en una próxima etapa.

Pádel & Gol

Sistema de Gestión de Reservas Deportivas

Informe de desarrollo — TP4

Morán Santiago / Fakiani Octavio | AW2 — IES 21 — 2026

¿Qué se hizo en este TP?

En el TP4 se completó el sistema de autenticación (registro, login y logout) usando JWT, hashing de contraseñas con **bcryptjs** y cookies **httpOnly** firmadas. También se reestructuró todo el proyecto al patrón de módulos por entidad pedido por la cátedra, y se sumó el módulo de reservas completo, que en el TP3 había quedado pendiente de migrar a la base de datos.

Además, se habilitó que cualquier visitante pueda reservar una cancha sin necesidad de iniciar sesión, que es la idea central de un sistema de gestión de reservas (un cliente no tiene por qué tener una cuenta para reservar un turno). El alta, baja y modificación de canchas, en cambio, sigue protegida y solo la puede hacer un administrador logueado.

Autenticación y seguridad

- Las contraseñas se hashean con **bcryptjs** antes de guardarse; nunca se guarda la contraseña en texto plano.
- Al iniciar sesión se genera un **JWT** firmado y se guarda en una cookie **httpOnly** y firmada (no se puede leer ni manipular desde JavaScript del navegador, lo que reduce el riesgo de robo de sesión por XSS).
- El middleware **comprobarToken** valida el JWT en cada pedido a una ruta protegida. Si el token no es válido o no existe, responde 401 (si es la API) o redirige a login.html (si es una vista).
- Las claves de firma (FIRMA_COOKIE, FIRMA_JWT) y la cadena de conexión a la base viven en variables de entorno (.env), que no se sube al repositorio.

Estructura del proyecto (patrón de módulos)

Se dejó de usar la carpeta única de models/controllers/routes del TP3 y se pasó a un módulo por entidad, cada uno con su modelo, controlador, rutas y vista:

- **modulos/canchas/**: CRUD de canchas (igual que en TP3, reubicado al nuevo patrón).
- **modulos/usuarios/**: registro, login y logout.
- **modulos/reservas/**: CRUD de reservas, separado en rutas administrativas (protegidas) y rutas públicas para la web.
- **middlewares/comprobarToken.mjs**: protege el panel de administración y las rutas de escritura.

Endpoints nuevos de la API

Usuarios:

- POST /registrar: crea un usuario nuevo.

- POST /login: valida usuario y contraseña, devuelve la cookie con el JWT.
- POST /logout: borra la cookie de sesión.

Reservas para administración (protegidas con comprobarToken):

- GET /api/reservas y GET /api/reservas/:id
- POST /api/reservas, PUT /api/reservas/:id, DELETE /api/reservas/:id

Reservas públicas, para la web (sin necesidad de sesión):

- GET /api/web/reservas y GET /api/web/reservas/:id: listado y detalle.
- POST /api/web/reservas: crea una reserva nueva desde reservas.html.

Validación de conflicto de horarios

Antes de crear una reserva, el controlador consulta si ya existe una reserva activa (con estado distinto de “cancelada”) para la misma cancha, fecha y hora. Si existe, devuelve un error 409 y no permite la doble reserva. Así se evita el problema que la propuesta original identificaba como uno de los principales motivos para reemplazar las planillas y los chats de WhatsApp.

¿Cómo se corre el proyecto?

Hay que crear un archivo .env con la conexión a la base (DATABASE_URL), el puerto y las claves de firma (FIRMA_COOKIE, FIRMA_JWT). Después, dentro de la carpeta del proyecto:

- npm install
- npm run db:init (crea las tablas canchas, usuarios y reservas si no existen)
- npm run dev

Y se abre el navegador en <http://localhost:3000>

Con esto, el sistema queda funcional de punta a punta: un visitante puede ver las canchas, reservar un turno sin necesidad de crear una cuenta, y un administrador logueado puede gestionar las canchas del complejo desde el panel.